

# Lab Experiment 14: List, Create, Delete, and Set Partition Types for Primary, Extended, and Logical Partitions

## Objective

To understand and perform basic disk partition operations in Linux such as:

1. Listing existing partitions
2. Creating and deleting partitions
3. Setting partition types (Primary, Extended, Logical)

## Background

A **disk partition** divides a physical storage device (like a hard drive) into separate sections called **partitions**. Each partition can store a separate filesystem (for example, /, /home, /boot, or swap area).

Linux typically identifies disks as:

- /dev/sda, /dev/sdb — Physical or virtual drives
- /dev/sda1, /dev/sda2, etc. — Partitions on those drives

## I. Types of Partitions

| Type               | Description                                                                                                       | Example              |
|--------------------|-------------------------------------------------------------------------------------------------------------------|----------------------|
| Primary Partition  | The main partition used to boot or install the OS. You can have <b>up to 4 primary partitions</b> on an MBR disk. | /dev/sda1, /dev/sda2 |
| Extended Partition | A special partition that acts as a <b>container</b> for logical partitions.                                       | /dev/sda4            |
| Logical Partition  | Subdivisions inside the extended partition. Used to overcome the 4-partition limit.                               | /dev/sda5, /dev/sda6 |

**Note:** Modern systems use **GPT** (GUID Partition Table) which removes these limits, but for learning purposes, we focus on **MBR** concepts.

## II. Commands and Tools Used

| Command                    | Purpose                                                                |
|----------------------------|------------------------------------------------------------------------|
| <code>lsblk</code>         | Lists all disks and partitions in tree format                          |
| <code>fdisk</code>         | Interactive tool for managing MBR partitions                           |
| <code>parted</code>        | Tool for GPT/MBR partition tables (alternative to <code>fdisk</code> ) |
| <code>df -h</code>         | Shows mounted partitions and their usage                               |
| <code>sudo fdisk -l</code> | Lists detailed disk partition information                              |

## III. Listing Existing Partitions

### 1 View All Available Disks

```
lsblk
```

Output Example:

| NAME | MAJ:MIN | RM | SIZE | RO | TYPE | MOUNTPOINT |
|------|---------|----|------|----|------|------------|
| sda  | 8:0     | 0  | 50G  | 0  | disk |            |
| sda1 | 8:1     | 0  | 20G  | 0  | part | /          |
| sda2 | 8:2     | 0  | 10G  | 0  | part | /home      |
| sda3 | 8:3     | 0  | 2G   | 0  | part | [SWAP]     |

### 2 Show Disk and Partition Details

```
sudo fdisk -l
```

This lists all available disks, their sizes, and partition types (Linux, swap, etc.).

## IV. Creating a New Partition

**Important:** For classroom/lab use on VirtualBox, practice only on a **secondary virtual disk** (e.g., `/dev/sdb`) — not the main system disk `/dev/sda`.

### 1 Create a New Virtual Disk (if using VirtualBox)

- Go to VirtualBox → Settings → Storage → Add Hard Disk → Choose 2 GB (for testing).
- Boot the system again and check:

```
lsblk
```

You'll see something like:

```
sdb      8:16    0    2G    0 disk
```

## 2 Start Partitioning with fdisk

```
sudo fdisk /dev/sdb
```

Now, you will enter **interactive mode** inside **fdisk**. The prompt changes to:

```
Command (m for help):
```

## 3 Useful Commands Inside fdisk

| Key | Function | Description                                |
|-----|----------|--------------------------------------------|
| m   | Help     | Displays available commands                |
| p   | Print    | Shows current partition table              |
| n   | New      | Creates a new partition                    |
| d   | Delete   | Deletes an existing partition              |
| t   | Type     | Changes partition type (Linux, swap, etc.) |
| w   | Write    | Saves changes and exits                    |
| q   | Quit     | Exit without saving changes                |

## 4 Create a Primary Partition

Steps:

1. Type **n** → press **Enter**
2. Choose **p** for primary
3. Choose partition number (e.g., 1)
4. Accept defaults or specify size (e.g., +500M)
5. Type **p** to verify the partition table
6. Type **w** to save changes

## 5 Create an Extended Partition

Repeat:

1.  $n \rightarrow e$  for extended
2. Choose partition number (e.g., 2)
3. Accept size (e.g., +1G)
4. Verify with **p**
5. Save with **w**

## 6 Create Logical Partitions Inside the Extended Partition

1.  $n \rightarrow 1$  for logical
2. Specify size (e.g., +200M)
3. Save using **w**

# V. Changing Partition Type

## 1 View Current Partition Type

Inside fdisk:

```
Command (m for help): p
```

You'll see a column **Type** (e.g., "Linux").

## 2 Change Partition Type

```
t
```

Then type the **hex code** (for example):

- 82 = Linux Swap
- 83 = Linux Filesystem
- 07 = HPFS/NTFS (Windows)
- 0b = FAT32

Then verify with:

```
p
```

Finally save:

```
w
```

## VI. Deleting a Partition

Inside fdisk:

```
sudo fdisk /dev/sdb
```

Then type:

```
d
```

Select partition number → confirm → **w** to save.

Verify:

```
lsblk
```

The partition should be gone.

## VII. Verifying the Changes

After creating or deleting partitions:

```
lsblk  
sudo fdisk -l
```

These commands will show the updated partition layout.

## Learning Outcome

After completing this experiment, students will be able to:

- Understand the difference between **primary**, **extended**, and **logical** partitions.
- List existing partitions and their details using **lsblk** and **fdisk -l**.
- Create, delete, and modify partitions safely on a secondary disk.
- Identify partition types and their hexadecimal codes.

# Lab Experiment 15: Create, Mount, Unmount, and Use ext2, ext3, and ext4 File Systems

## Objective

To learn how to:

1. **Create partitions** and format them with **ext2**, **ext3**, and **ext4** file systems.
2. **Mount and unmount** these file systems.
3. **Access and use** files stored within mounted partitions.

## Background

In Linux, the **file system** defines how data is stored and organized on a disk or partition. Each partition must be **formatted** with a file system before it can be used.

Common Linux file systems:

| File System | Description                                                                          |
|-------------|--------------------------------------------------------------------------------------|
| <b>ext2</b> | Second Extended File System — older, no journaling, used in early Linux.             |
| <b>ext3</b> | Third Extended File System — adds <b>journaling</b> (better recovery after crashes). |
| <b>ext4</b> | Fourth Extended File System — modern, supports large files and improved performance. |

## I. Preliminary Concepts

| Term               | Meaning                                                                                  |
|--------------------|------------------------------------------------------------------------------------------|
| <b>Format</b>      | Prepare a partition with a file system.                                                  |
| <b>Mount</b>       | Attach the formatted partition to a directory so it can be accessed.                     |
| <b>Unmount</b>     | Detach the partition safely from the directory.                                          |
| <b>Mount point</b> | A folder where a device or partition is made accessible (e.g., <code>/mnt/data</code> ). |

## II. Tools and Commands Used

| Command                    | Purpose                                        |
|----------------------------|------------------------------------------------|
| <code>lsblk</code>         | Lists block devices (disks/partitions).        |
| <code>sudo fdisk -l</code> | Lists partitions with details.                 |
| <code>mkfs</code>          | Creates (formats) a file system.               |
| <code>mount</code>         | Mounts a device or partition to a directory.   |
| <code>umount</code>        | Unmounts a device or partition.                |
| <code>df -h</code>         | Displays mounted file systems and their usage. |

## III. Step-by-Step Procedure

### Note for Students:

Always practice on a **secondary virtual disk** (like `/dev/sdb`) in **VirtualBox**, never on `/dev/sda` (the system disk).

### Step 1: Identify Available Disks

```
lsblk
```

Example Output:

| NAME | MAJ:MIN | RM | SIZE | RO | TYPE | MOUNTPOINT |
|------|---------|----|------|----|------|------------|
| sda  | 8:0     | 0  | 50G  | 0  | disk |            |
| sda1 | 8:1     | 0  | 50G  | 0  | part | /          |
| sdb  | 8:16    | 0  | 2G   | 0  | disk |            |

If `/dev/sdb` doesn't exist, add a new **2 GB virtual disk** through VirtualBox → Settings → Storage.

### Step 2: Create a Partition

Use `fdisk` on the secondary disk:

```
sudo fdisk /dev/sdb
```

Inside `fdisk`:

```
n  # create new partition
p  # primary
1  # partition number
+500M  # size
w  # write changes and exit
```

Check new partition:

```
lsblk
```

You'll now see `/dev/sdb1`.

### Step 3: Create File Systems

(a) **Create ext2 file system**

```
sudo mkfs.ext2 /dev/sdb1
```

(b) **Create ext3 file system** First, create another partition (e.g., `/dev/sdb2`), then:

```
sudo mkfs.ext3 /dev/sdb2
```

(c) **Create ext4 file system** Create one more partition (e.g., `/dev/sdb3`), then:

```
sudo mkfs.ext4 /dev/sdb3
```

### Step 4: Create Mount Points

Mount points are simply directories that provide access to the partition.

```
sudo mkdir /mnt/ext2
sudo mkdir /mnt/ext3
sudo mkdir /mnt/ext4
```

### Step 5: Mount the File Systems

```
sudo mount /dev/sdb1 /mnt/ext2
sudo mount /dev/sdb2 /mnt/ext3
sudo mount /dev/sdb3 /mnt/ext4
```

Verify:

```
df -h
```

You'll see all three mounted partitions listed with their file system types.

### Step 6: Use the Mounted File Systems

Now that they're mounted, you can **create files** and **directories** inside them like normal folders.



```
sudo touch /mnt/ext2/test_ext2.txt
sudo touch /mnt/ext3/test_ext3.txt
sudo touch /mnt/ext4/test_ext4.txt

ls /mnt/ext2
ls /mnt/ext3
ls /mnt/ext4
```

You've successfully created and used files inside mounted partitions.

## Step 7: Unmount the File Systems

Unmount them safely using:

```
sudo umount /mnt/ext2
sudo umount /mnt/ext3
sudo umount /mnt/ext4
```

Verify:

```
df -h
```

The mounted file systems should no longer appear.

## IV. Verify File System Type

You can verify the file system format using:

```
sudo blkid
```

Example Output:

```
/dev/sdb1: UUID="b43f-12d9" TYPE="ext2"
/dev/sdb2: UUID="c32a-45bc" TYPE="ext3"
/dev/sdb3: UUID="f54d-9a01" TYPE="ext4"
```

## Learning Outcome

After completing this experiment, students will be able to:

- Differentiate between **ext2**, **ext3**, and **ext4** file systems.
- Format partitions using different file systems.
- Mount and unmount partitions safely.
- Create and access files within mounted file systems.
- Verify and confirm file system types in Linux.

# Lab Experiment 16: Create and Manage Access Control Lists (ACLs)

## Objective

To understand, create, and manage **Access Control Lists (ACLs)** in Linux, which provide **fine-grained file permission control** beyond standard UGO/RWX permissions.

## Background

In Linux, standard permissions only allow controlling access for:

- One user (owner)
- One group
- Others (everyone else)

To give different permissions to multiple users or groups on the same file, Linux provides **Access Control Lists (ACLs)**.

## I. What are ACLs?

| Term                      | Description                                                        |
|---------------------------|--------------------------------------------------------------------|
| ACL (Access Control List) | A list of permission rules applied to a file or directory.         |
| User ACL                  | Grants access to a specific user other than the owner.             |
| Group ACL                 | Grants access to a specific group.                                 |
| Mask                      | Defines the maximum allowed permissions for all users and groups.  |
| Default ACL               | Automatically applies permissions to new files inside a directory. |

## II. Commands Used

| Command              | Description                                    |
|----------------------|------------------------------------------------|
| <code>getfacl</code> | Displays current ACLs for a file or directory. |
| <code>setfacl</code> | Adds, modifies, or removes ACL permissions.    |
| <code>ls -l</code>   | Displays standard permissions.                 |
| <code>chmod</code>   | Sets normal UGO/RWX permissions (baseline).    |

## III. Checking ACL Support

Most modern Linux filesystems support ACLs. Verify using:

```
mount | grep acl
```

## IV. Step-by-Step Procedure

### Step 1: Create a Directory and File

```
mkdir acl_demo  
cd acl_demo  
touch file1.txt
```

### Step 2: Create Two Users (Optional for Testing)

```
sudo useradd user1  
sudo useradd user2  
sudo passwd user1  
sudo passwd user2
```

### Step 3: Check Default Permissions

```
ls -l file1.txt
```

### Step 4: Grant ACL Permission to a User

Give user1 read/write permissions:

```
sudo setfacl -m u:user1:rw file1.txt
```

### Step 5: Verify ACL

```
getfacl file1.txt
```

### Step 6: Add Another User's ACL

Give user2 read-only access:

```
sudo setfacl -m u:user2:r file1.txt
```

### Step 7: Test Access for Both Users

As user1:

```
sudo su user1  
echo "Hello from user1" >> file1.txt
```

As user2:

```
sudo su user2
cat file1.txt          # Works
echo "Test" >> file1.txt  # Permission denied
```

### Step 8: Remove ACL Entry

```
sudo setfacl -x u:user2 file1.txt
```

## VII. Reset (Remove) All ACLs

```
sudo setfacl -b file1.txt
```

## VIII. Summary Table

| Task            | Command                                     | Description                   |
|-----------------|---------------------------------------------|-------------------------------|
| Add user ACL    | <code>setfacl -m u:user1:rw file.txt</code> | Give user1 read/write access. |
| View ACL        | <code>getfacl file.txt</code>               | Display ACLs for file.        |
| Remove user ACL | <code>setfacl -x u:user1 file.txt</code>    | Delete user1's ACL rule.      |
| Remove all ACLs | <code>setfacl -b file.txt</code>            | Clear all ACL settings.       |

# Lab Experiment 17: Diagnose and Correct File Permission Problems

## Objective

To learn how to **identify**, **diagnose**, and **correct** file permission and ownership problems in Linux using appropriate commands such as `ls -l`, `chmod`, `chown`, and `getfacl`.

## Background

In Linux, every file and directory has specific **permissions** and **ownership** that control who can:

- Read (**r**)
- Write (**w**)
- Execute (**x**)

Sometimes, incorrect permissions or ownership settings cause “Permission denied” errors or restrict users from accessing important files. This experiment teaches how to **find the cause** of such errors and **fix them safely**.

## I. Key Concepts Recap

| Concept     | Description                                                                             |
|-------------|-----------------------------------------------------------------------------------------|
| User (u)    | File owner (usually the creator)                                                        |
| Group (g)   | Users belonging to the same group                                                       |
| Others (o)  | All other users                                                                         |
| Read (r)    | Allows reading file or listing directory                                                |
| Write (w)   | Allows modifying file or adding/removing files in directory                             |
| Execute (x) | Allows running file or entering directory                                               |
| Ownership   | Controlled by <code>chown</code> (user) and <code>chgrp</code> (group)                  |
| ACL         | Provides advanced permission control (via <code>setfacl</code> , <code>getfacl</code> ) |

## II. Common File Permission Problems

| Problem                          | Possible Cause                     | Error Example                           |
|----------------------------------|------------------------------------|-----------------------------------------|
| File cannot be opened or read    | Missing read permission (r)        | cat: permission denied                  |
| File cannot be edited            | Missing write permission (w)       | nano: permission denied                 |
| Script/program won't run         | Missing execute permission (x)     | bash: ./script.sh:<br>Permission denied |
| User cannot access directory     | No execute permission on directory | cd: permission denied                   |
| User cannot modify shared folder | Ownership or ACL issues            | Files not writable                      |
| File accessible by wrong user    | Incorrect owner/group              | Shared incorrectly between users        |

## III. Commands Used for Troubleshooting

| Command | Purpose                         |
|---------|---------------------------------|
| ls -l   | Check permissions and ownership |
| ls -ld  | Check directory permissions     |
| chmod   | Change permissions              |
| chown   | Change file owner               |
| chgrp   | Change group ownership          |
| getfacl | View ACL permissions            |
| setfacl | Modify ACL permissions          |

## IV. Step-by-Step Procedure

### Step 1: Create a Test File and Directory

```
mkdir perm_demo  
cd perm_demo  
touch testfile.txt
```

Check current permissions:

```
ls -l
```

Example output:

```
-rw-r--r-- 1 masood masood 0 Nov 7  testfile.txt
```

This means:

- Owner: read/write

- Group: read
- Others: read

## Step 2: Simulate Permission Problems

(a) Remove read permission:

```
chmod 377 testfile.txt  
cat testfile.txt
```

Output:

```
cat: testfile.txt: Permission denied
```

(b) Remove write permission:

```
chmod 577 testfile.txt  
echo "hello" >> testfile.txt
```

Output:

```
bash: testfile.txt: Permission denied
```

## Step 3: Diagnose the Problem

```
ls -l  
ls -ld
```

Check ACLs:

```
getfacl testfile.txt
```

## Step 4: Correct the Problem

(a) Restore read permissions:

```
chmod 777 testfile.txt  
cat testfile.txt
```

(b) Restore write permission:

```
chmod 777 testfile.txt  
echo "hello" >> testfile.txt
```

## Step 5: Verify Changes

```
ls -l
```

Confirm the correct **rw**x permissions and ownership.